

TEAM 9: REFACTORING

COURSE: SOEN 6441 (WINTER 24)

INSTRUCTOR- JOEY PAQUET

TEAM MEMBERS:

1. Jayasurya Pazhani
2. Jayanth Apagundi
3. Shuvanidhi Suresh
4. Shahul Hameed P T S
5. Vinisha Kora Venkatesalu

Potential Refactoring Targets:

The following list of refactoring targets has been taken mainly from the new requirements established in build 3 and based on pain points and inconsistencies encountered during the development of build 2.

1. Implemented the MapReaderAdapter and MapWriterAdapter, using Adapter pattern (Observer pattern).
2. Implementing the Strategy Pattern: Framework and Execution (State pattern).
3. Streamlining Metadata Handling: Centralizing Parsing and Writing Methods in MapFileReader and MapFileWriter.
4. Enhancing exception Handling.
5. Serializable Interface.
6. Streamlining Parsing and Writing Operations.
7. Improve Code Coverage with Additional Tests.
8. Optimizing loops for better performance.
9. Organize code into logical sections, and group related methods together.
10. Simplify complex conditional logic.
11. Encapsulate data members with appropriate access modifiers to enforce data.
12. Decompose large methods or classes into smaller, cohesive components to improve.
13. code organization.

- 14.integrity and information hiding
- 15.Method Refactoring.
- 16.Code Structure and Organization.

Actual Refactoring Targets:

1.Implemented the MapReaderAdapter and MapWriterAdapter, using Adapter pattern.

Before:

The code from the build 2 used Observer pattern , the GameEngine class had a Phase field representing the current phase of the game. Each instead of directly printing message to the console,a method setD_playerLog is called to set a log message. This change likely indicates a broader architectural decision to manage and handle log messages more systematically within the codebase.

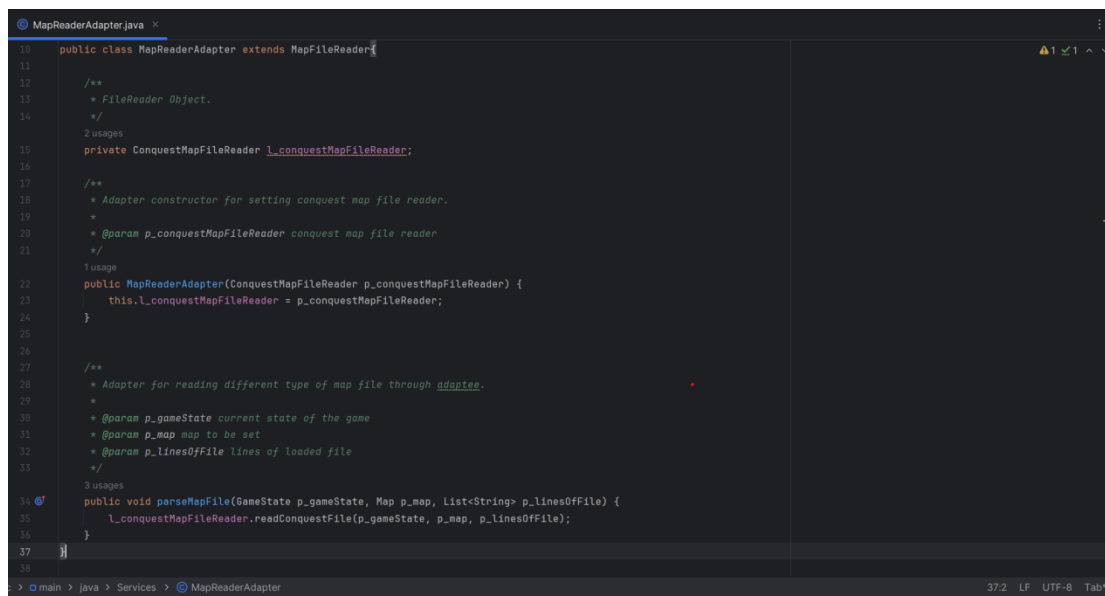
After:

In MapReaderAdapter and MapWriterAdapter:

- The adapter classes extend the target interfaces (MapFileReader and MapFileWriter).
- They hold an instance of the adaptee classes (ConquestMapFileReader and ConquestMapFileWriter) through composition.
- They delegate the specific tasks to the adaptee classes, thus adapting their interfaces to match the expected interfaces.

The adapter pattern allows users to use a interface of an existing class to be used as an other interface

List of TestCases: testReadMapFile – MapFileReaderTest, testReadConquestFile – ConquestMapFileReaderTest, testEditMap - ConquestMapFileReaderTest



```
10 public class MapReaderAdapter extends MapFileReader {
11
12     /**
13      * FileReader Object.
14      */
15     2 usages
16     private ConquestMapFileReader l_conquestMapFileReader;
17
18     /**
19      * Adapter constructor for setting conquest map file reader.
20      *
21      * @param p_conquestMapFileReader conquest map file reader
22      */
23     1 usage
24     public MapReaderAdapter(ConquestMapFileReader p_conquestMapFileReader) {
25         this.l_conquestMapFileReader = p_conquestMapFileReader;
26     }
27
28     /**
29      * Adapter for reading different type of map file through adaptee.
30      *
31      * @param p_gameState current state of the game
32      * @param p_map map to be set
33      * @param p_linesOfFile lines of loaded file
34      */
35     3 usages
36     public void parseMapFile(GameState p_gameState, Map p_map, List<String> p_linesOfFile) {
37         l_conquestMapFileReader.readConquestFile(p_gameState, p_map, p_linesOfFile);
38     }
39 }
```

2. Implementing the Strategy Pattern: Framework and Execution

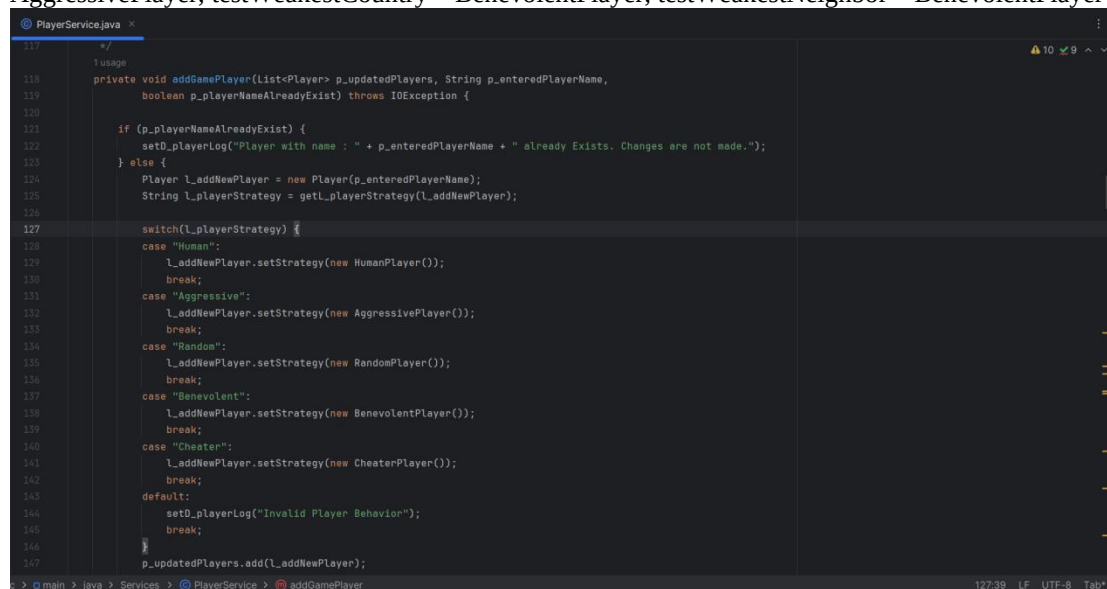
Before:

The code forms the build 2 used State pattern, the `GameEngine` class had a `Phase` field representing the current phase of the game. Each phase is encapsulated within its own class (`StartupPhase`, `IssueOrderPhase`, `OrderExecutionPhase`, etc.), implementing a common `Phase` interface or abstract class. The `GameEngine` class delegates phase-specific behavior to the current phase object.

After:

- **Strategy Interface:** A new interface named `PlayerBehaviorStrategy` is used as a strategy interface.
- **Strategy Attribute:** The `Player` class has a new attribute `d_playerBehaviorStrategy` of type `PlayerBehaviorStrategy`. This attribute holds an instance of a strategy that determines the player's behavior.
- **Setting Strategy:** The `setStrategy()` method has been added to the `Player` class, allowing the strategy for the player to be set dynamically.
- **Using Strategy:** The `getPlayerOrder()` method uses the strategy pattern to get the order. It delegates the order creation to the strategy assigned to the player.

List of TestCase: `testOrderCreation` – `Aggressive`, `Random`, `Cheater`, and `Benevolent`, `testStrongestCountry` – `AggressivePlayer`, `testWeakestCountry` – `BenevolentPlayer`, `testWeakestNeighbor` – `BenevolentPlayer`



```
118 private void addGamePlayer(List<Player> p_updatedPlayers, String p_enteredPlayerName,
119 boolean p_playerNameAlreadyExist) throws IOException {
120
121     if (p_playerNameAlreadyExist) {
122         setD_playerLog("Player with name : " + p_enteredPlayerName + " already Exists. Changes are not made.");
123     } else {
124         Player l_addNewPlayer = new Player(p_enteredPlayerName);
125         String l_playerStrategy = getL_playerStrategy(l_addNewPlayer);
126
127         switch(l_playerStrategy) {
128             case "Human":
129                 l_addNewPlayer.setStrategy(new HumanPlayer());
130                 break;
131             case "Aggressive":
132                 l_addNewPlayer.setStrategy(new AggressivePlayer());
133                 break;
134             case "Random":
135                 l_addNewPlayer.setStrategy(new RandomPlayer());
136                 break;
137             case "Benevolent":
138                 l_addNewPlayer.setStrategy(new BenevolentPlayer());
139                 break;
140             case "Cheater":
141                 l_addNewPlayer.setStrategy(new CheaterPlayer());
142                 break;
143             default:
144                 setD_playerLog("Invalid Player Behavior");
145                 break;
146         }
147         p_updatedPlayers.add(l_addNewPlayer);
148     }
```

3.Streamlining Metadata Handling: Centralizing Parsing and Writing Methods in MapFileReader and MapFileWriter.

Before:

Previously in Build 2, we used map services both to read and write a mapfile and we also used only one type of map file along with that it also provides functionalities for editing, saving, and validating maps. It encompasses a broad range of operations related to map manipulation.

After:

In Build 3, `MapFileReader` is primarily focused on parsing map files into game state objects. Unlike `MapService`, its responsibilities seem narrowly defined, emphasizing map file reading, and parsing without including editing or validation functionalities. This method also Implements the `Serializable` interface, hinting at potential use cases involving serialization/deserialization of the reader itself, possibly for state saving, networking, or other purposes not directly related to map data parsing.

The parsing methods in `MapFileWriter` are organized to handle the writing of country, border, and continent metadata separately (`writeCountryAndBoarderMetaData`, `writeContinentMetadata`).

List of TestCases: testReadMapFile – MapFileReaderTest, testReadConquestFile -ConquestMapFileReaderTest, testEditMap – MapServiceTest, testEditContinentAdd - MapServiceTest

```

MapFileReader.java
75 @ public List<Continent> parseContinentsMetaData(List<String> p_continentList) {
76     int l_continentId = 1;
77     List<Continent> l_continents = new ArrayList<>();
78
79     for (String cont : p_continentList) {
80         String[] l_metaData = cont.split(regex: "[|]");
81         l_continents.add(new Continent(l_continentId, l_metaData[0], Integer.parseInt(l_metaData[1])));
82         l_continentId++;
83     }
84     return l_continents;
85 }
86
87 /**
88  * The parseCountriesMetaData method parse extracted country and border data of
89  * map file.
90  *
91  * @param p_countriesList includes country data in list from map file.
92  * @return List of processed country meta data.
93  */
94 @ public List<Country> parseCountriesMetaData(List<String> p_countriesList) {
95     List<Country> l_countriesList = new ArrayList<>();
96
97     for (String country : p_countriesList) {
98         String[] l_metaDataCountries = country.split(regex: "[|]");
99         l_countriesList.add(new Country(Integer.parseInt(l_metaDataCountries[0]), l_metaDataCountries[1],
100             Integer.parseInt(l_metaDataCountries[2])));
101     }
102     return l_countriesList;
103 }
104
105

```

4.Enhancing Exeption Handling:

Before:

In the "before" version, we did not have tournament instead we had only single player mode where multiple players cannot play at the same time.

After:

In build 3 we have added INVALID_COMMAND_TOURNAMENT_MODE to validate the command to start a new tournament and command related to tournament.

List of TestCases: testValidTournament – TournamentTest, testInvalidPlayerStrategiesArgs – TournamentTest, testInvalidNoOfGamesArgs – TournamentTest, testInvalidMapArgs - TournamentTest

```

Tournament.java
147 if (p_operation.equalsIgnoreCase(anotherString: "P")) {
148     return parseStrategyArguments(p_gameState, p_argument, p_gameEngine);
149 }
150 if (p_operation.equalsIgnoreCase(anotherString: "G")) {
151     return parseNoOfGameArgument(p_argument, p_gameEngine);
152 }
153 if (p_operation.equalsIgnoreCase(anotherString: "D")) {
154     return parseNoOfTurnsArguments(p_argument, p_gameEngine);
155 }
156 throw new InvalidCommand(ApplicationConstants.INVALID_COMMAND_TOURNAMENT_MODE);
157 }
158
159
160 @ private void setTournamentPlayers(GameEngine p_gameEngine, String[] p_listOfPlayerStrategies,
161     List<Player> p_listOfPlayers, List<Player> p_playersInTheGame) {
162     for (String l_strategy : p_listOfPlayerStrategies) {
163         for (Player l_pl : p_listOfPlayers) {
164             if (l_pl.getPlayerBehaviorStrategy().getPlayerBehavior().equalsIgnoreCase(l_strategy)) {
165                 p_playersInTheGame.add(l_pl);
166                 p_gameEngine.setLogGameEngineLog(p_gameEngineLog: "Player: " + l_pl.getPlayerName() + " with strategy: " + l_strategy
167                     + " has been added in tournament.", p_logType: "effect");
168             }
169         }
170     }
171 }
172
173
174 @ private boolean parseMapArguments(String p_argument, GameEngine p_gameEngine) throws InvalidMap {
175     String[] l_listOfMapFiles = p_argument.split(regex: "[|]");
176     int l_mapFilesSize = l_listOfMapFiles.length;
177

```

5. Serializable Interface:

The Diplomacy class now implements the Serializable interface, allowing instances of the class to be serialized and deserialized, which can be useful for various operations like saving game states.

Before:

```
Diplomacy.java x
1 package Models;
2
3 import Services.PlayerService;
4
5 /**
6  * Represents a diplomacy card in the game.
7  */
8
9 public class Diplomacy implements Card {
10
11     Player d_IssuingPlayer;
12
13     String d_targetPlayer;
14
15     String d_orderExecutionLog;
16
17     /**
18      * Initializes a new instance of the Diplomacy class.
19      *
20      * @param p_targetPlayer The target player for the diplomacy.
21      * @param p_IssuingPlayer The player issuing the diplomacy.
22      */
23 }
```

After:

```
Diplomacy.java x
1 package Models;
2
3 import Services.PlayerService;
4 import java.io.Serializable;
5
6 /**
7  * Represents a diplomacy card in the game.
8  */
9
10 public class Diplomacy implements Card , Serializable{
11
12     Player d_IssuingPlayer;
13
14     String d_targetPlayer;
15
16     String d_orderExecutionLog;
17
18     /**
19      * Initializes a new instance of the Diplomacy class.
20      *
21      * @param p_targetPlayer The target player for the diplomacy.
22      * @param p_IssuingPlayer The player issuing the diplomacy.
23      */
24 }
```